Лабораторная работа №8. Приемочное тестирование и BDD (Behavior-Driven Development)

1. Цель работы

Изучить методологию приемочного тестирования, освоить подход Behavior-Driven Development (BDD) и инструменты для спецификации требований на понятном заказчику языке. Получить практические навыки написания сценариев на языке Gherkin и их автоматизации с использованием специализированных фреймворков.

2. Теоретическая часть

2.1. Приемочное тестирование (Acceptance Testing)

Приемочное тестирование — это финальная стадия тестирования, которая проводится для определения готовности системы к эксплуатации и соответствия её бизнес-требованиям заказчика. Основная цель — подтвердить, что система делает то, что нужно пользователю, в реальных условиях.

Виды приемочного тестирования:

1. **Альфа-тестирование (Alpha Testing):** Проводится внутри организации-разработчика, но с участием ограниченной группы пользователей или специальной тестовой группой.
2. **Бета-тестирование (Beta Testing):** Проводится реальными пользователями в их рабочей среде. Пользователи сообщают о найденных проблемах и впечатлениях.
3. **Операционное приемочное тестирование (Operational Acceptance Testing — OAT):** Проверка готовности инфраструктуры, процедур резервного копирования, восстановления и технической поддержки.
4. **Приемочное тестирование по контракту (Contract Acceptance Testing):** Проверка выполнения условий, прописанных в контракте на разработку.

Критерии приемки (Definition of Done / Acceptance Criteria) — это набор условий, которым должно удовлетворять разработанное решение, чтобы быть принятым заказчиком. Именно эти критерии ложатся в основу тестовых сценариев .

2.2. Behavior-Driven Development (BDD)

BDD (Разработка, управляемая поведением) — это гибкая методология разработки, которая развивает идеи TDD (Test-Driven Development) и приемочного тестирования . Ключевое отличие — BDD фокусируется

на **поведении** системы с точки зрения бизнеса и пользователя, а не на технических деталях реализации .

Основные принципы BDD :

- **Сотрудничество:** BDD поощряет тесное взаимодействие между разработчиками, тестировщиками, бизнес-аналитиками и заказчиками.
- **Единый язык (Ubiquitous Language):** Использование общего, понятного всем участникам проекта языка для описания требований. Это устраняет недопонимание между "технарями" и "бизнесом" .
- **Акцент на поведение:** Вместо того чтобы думать "как протестировать этот метод", мы думаем "как система должна вести себя в этой ситуации".

BDD не заменяет TDD, а дополняет его, работая на более высоком уровне — уровне требований.

2.3. Язык Gherkin

Gherkin — это предметно-ориентированный язык (DSL), который используется в BDD для описания поведения программного обеспечения в формате "текстовых историй" (plain text). Файлы с описанием имеют расширение .feature .

Ключевые ключевые слова Gherkin:

- Feature (Функция): Описывает тестируемую функцию или возможность. Содержит название и, опционально, краткое описание.
- Scenario (Сценарий): Конкретный пример использования функции. Каждый сценарий состоит из последовательности шагов.
- Given (Дано): Описывает предварительное условие — контекст, в котором начинается сценарий (например, состояние системы, данные).
- When (Когда): Описывает действие или событие, которое инициирует сценарий (например, нажатие кнопки, отправка формы).
- Then (Тогда): Описывает ожидаемый результат или проверку (assertion).
- And (И), But (Но): Служат для расширения предыдущих шагов (Given, When, Then) и делают сценарий более читаемым .
- Scenario Outline (Структура сценария) и Examples (Примеры): Используются для создания параметризованных сценариев, которые запускаются несколько раз с разными наборами тестовых данных .

Пример Gherkin-сценария :

```
gherkin
Feature: Управление запасами при возвратах
```

```
Scenario: Товары, возвращенные для возмещения, должны добавляться в инвентарь.
  Given что покупатель ранее купил у меня черный свитер
    And у меня есть три черных свитера в инвентаре
  When он возвращает черный свитер для возмещения
  Then у меня должно быть четыре черных свитера в инвентаре
```

2.4. Инструменты для BDD

BDD-фреймворки связывают человеко-читаемые сценарии на Gherkin с исполняемым кодом. Они ищут соответствие между шагами в .feature файле и определениями шагов (step definitions), написанными на языке программирования .

- **Для .NET (C#): SpecFlow** — наиболее популярный фреймворк, тесно интегрирующийся с Visual Studio. Позволяет писать шаги на C# и использовать любые тестовые раннеры (MSTest, NUnit, xUnit) .
- **Для Python: Behave** — стабильный и хорошо документированный фреймворк для BDD на Python . Альтернативы: pytest-bdd, Lettuce .
- **Для Java: Cucumber-JVM** и **JBehave** .
- **Для JavaScript: Cucumber-JS**, Yadda .

Эти инструменты работают на одном принципе: Gherkin сценарий → Step Definitions → Код приложения/Автоматизации .

Материалы для самостоятельного изучения:

1. Что такое приёмочное тестирование (Acceptance Testing)? https://qarocks.ru/acceptance-testing/#beta-testing/field-testing.
2. Введение в BDD: https://habr.com/ru/companies/auriga/articles/808527.
3. BDD тестирование. Язык Gherkin. Фреймворк Cucumber: https://smartiqa.ru/blog/bdd_gherkin_cucumber.
4. Cucumber: превращаем тесты в понятные бизнес-сценарии – BDD подход: https://sky.pro/wiki/python/ponimaem-funktsiyu-enumerate-v-python-na-primere-koda/.
5. Исполняемая спецификация: SpecFlow от А до Я. https://habr.com/ru/articles/182032/.

3. Задание на лабораторную работу

Работа выполняется на основе приложения, разработанного в предыдущих лабораторных работах (система авторизации и связанные с ней функции).

Задание 1. Написание Gherkin-сценариев

Для своего варианта приложения составьте **3-5 сценариев** приемочного тестирования на языке Gherkin. Сценарии должны охватывать как позитивные, так и негативные пути, и быть оформлены в отдельных .feature файлах.

Требования к сценариям:

- Наличие заголовка Feature с кратким описанием.
- Использование ключевых слов Scenario, Given, When, Then, And.
- Как минимум один сценарий должен быть оформлен как Scenario Outline с таблицей Examples для проверки нескольких наборов данных.

Примеры возможных сценариев для системы авторизации:

- **Feature:** Смена пароля
  - *Сценарий 1:* Успешная смена пароля при соблюдении политик безопасности.
  - *Сценарий 2:* Попытка смены пароля на слишком короткий (нарушение политики) -> ожидаем сообщение об ошибке.
  - *Сценарий (Outline):* Проверка политики надежности пароля (разные комбинации в таблице Examples).
- **Feature:** Управление ролями пользователей
  - *Сценарий:* Администратор успешно меняет роль обычного пользователя.

Пример файла:

```
Feature: Смена пароля
  Как зарегистрированный пользователь
  Я хочу иметь возможность менять свой пароль
  Чтобы обеспечить безопасность моего аккаунта

  @positive
  Scenario: Успешная смена пароля
    Given пользователь "john_doe" авторизован в системе
    When пользователь меняет пароль с "oldPass123" на "NewStrongPass1"
    Then система должна показать сообщение "Пароль успешно изменен"
    And пользователь должен иметь возможность авторизоваться с новым паролем "NewStron
gPass1"

  @negative
  Scenario Outline: Смена пароля на несоответствующий политикам
    Given пользователь "john_doe" авторизован в системе
    When пользователь пытается установить пароль "<new_password>"
    Then система должна показать сообщение об ошибке "<error_message>"
```

```
Examples:
  | new_password | error_message                        |
  | "short"      | "Пароль должен быть не менее 8 символов" |
  | "nodigits"   | "Пароль должен содержать хотя бы одну цифру" |
```

Задание 2. Автоматизация приемочных сценариев

1. **Выбор инструментария:**

- В зависимости от стека технологий вашего приложения выберите BDD-фреймворк:
  - **C# (.NET):** Установите SpecFlow через NuGet в тестовый проект. Также установите необходимый адаптер тестов (например, SpecFlow.MsTest или SpecFlow.NUnit) и библиотеку для fluent-assertions (рекомендуется FluentAssertions) .
  - **Python:** Установите фреймворк behave (pip install behave) и, при необходимости, Selenium для UI-автоматизации .

2. **Реализация шагов (Step Definitions):**

- Создайте файл с определениями шагов (например, PasswordChangeSteps.cs для C# или steps.py для Python).
- Для каждого шага из ваших .feature файлов напишите соответствующий метод, который будет выполнять действие или проверку.
- Для взаимодействия с приложением используйте уже знакомые вам подходы: вызовы API, прямое обращение к сервисам (для модульного уровня) или UI-автоматизацию с помощью Selenium/FlaUI (как в ЛР №7).
- Для проверок (assertions) используйте соответствующие библиотеки .

3. **Запуск и отладка:**

- Запустите тесты. В первый раз они, скорее всего, упадут (если вы следуете принципам BDD, код еще не реализован, либо шаги написаны неверно) .
- Добейтесь успешного прохождения всех сценариев.

4. Содержание отчёта

Отчёт должен содержать:

Отчёт должен содержать:

1. **Цель работы.**
2. **Описание приемочных сценариев (Задание 1):**

- Полные листинги всех разработанных .feature файлов с комментариями, поясняющими логику каждого сценария.

- Скриншоты или текст с описанием, как эти сценарии соотносятся с функциональными требованиями к системе.

3. **Описание автоматизации (Задание 2):**

- Указание выбранного BDD-фреймворка и обоснование выбора.
- Листинги кода с определениями шагов (Step Definitions) для всех реализованных сценариев.
- Скриншоты успешного прохождения всех тестов (консольный вывод behave, окно Test Explorer в Visual Studio).
- Пример вывода результатов тестирования с указанием прошедших и упавших шагов .

4. **Выводы:**

- Сравните процесс написания тестов "с нуля" (как в ЛР №4-5) и через BDD-спецификации.
- Оцените, насколько понятными оказались Gherkin-сценарии для восприятия (представьте, что вы показываете их "заказчику").
- Опишите преимущества и возможные сложности, с которыми вы столкнулись при использовании BDD.

5. Контрольные вопросы

Базовые понятия

1. Что такое приемочное тестирование и чем оно отличается от системного?
2. Какие существуют виды приемочного тестирования?
3. Что такое критерии приемки (Acceptance Criteria) и как они связаны с тестами?
4. В чем заключается основная идея Behavior-Driven Development (BDD)?
5. Как BDD связан с TDD и чем они различаются?

Язык Gherkin и сценарии

6. Для чего нужен язык Gherkin? Перечислите его основные ключевые слова.
7. Объясните структуру BDD-сценария: Given, When, Then, And.
8. Что такое Scenario Outline и Examples? Для решения каких задач они применяются?
9. Кто является основным читателем Gherkin-сценариев и почему это важно?
10. Что такое "единый язык" (Ubiquitous Language) в контексте BDD?

Инструменты и реализация

11. Какие BDD-фреймворки для .NET/Python/Java вы знаете?

12. Что такое "step definitions" (определения шагов) и как они связаны с Gherkin-сценариями?

13. Как происходит процесс автоматизации BDD-сценария (от фичи до запуска теста)?

14. Можно ли использовать BDD для ручного тестирования? Если да, то как?

15. Какие библиотеки для "красивых" проверок (assertions) можно использовать вместе с SpecFlow?

Методология и лучшие практики

16. Какие три группы участников процесса разработки вовлечены в BDD и какова их роль?

17. Как обеспечить независимость BDD-тестов друг от друга?

18. Нужно ли автоматизировать все Gherkin-сценарии? Почему?

19. Как часто следует запускать автоматизированные приемочные тесты?

20. Какие метрики можно использовать для оценки качества приемочного тестирования?

6. Рекомендуемая литература и источники для самостоятельного изучения

6.1. Официальная документация и вводные руководства

- **SpecFlow. "Getting Started with SpecFlow" [Электронный ресурс] / SpecFlow Documentation.**
  - **Доступ:** https://docs.specflow.org/projects/specflow/en/latest/Getting-Started/Getting-Started.html
  - **Аннотация:** Официальное руководство по началу работы с SpecFlow для .NET. Содержит пошаговые инструкции по установке, созданию первого проекта, написанию фич и запуску тестов.
- **Behave. "Behave Documentation" [Электронный ресурс].**
  - **Доступ:** https://behave.readthedocs.io/en/latest/
  - **Аннотация:** Полная документация по фреймворку Behave для Python. Включает tutorial, описание API, примеры использования и лучшие практики.
- **Pluralsight. "Guided: Acceptance Testing" [Электронный ресурс] / Pluralsight Labs.**
  - **Доступ:** https://www.pluralsight.com/labs/codeLabs/guided-acceptance-testing

- **Аннотация:** Интерактивный лабораторный практикум, который проведет через весь процесс написания и автоматизации приемочных тестов для функции входа с использованием Python, Selenium и Behave .

- **Microsoft Community Hub. "BDD and SpecFlow for .NET" [Электронный ресурс] / Testing Spot Blog.**
  - **Доступ:** https://techcommunity.microsoft.com/t5/blogs/blogarticleprintpage/blog-id/TestingSpotBlog/article-id/117
  - **Аннотация:** Подробная статья от Microsoft, которая объясняет концепции BDD и предоставляет пошаговый туториал по установке и использованию SpecFlow в Visual Studio 2022 с примерами на C# .

6.2. Книги и углубленные материалы

- **Gundecha, U. "Selenium Testing Tools Cookbook" / Packt Publishing.**
  - **Доступ (глава):** https://www.packtpub.com/en-ie/product/selenium-testing-tools-cookbook-11-9781784392512/chapter/behavior-driven-development
  - **Аннотация:** Глава 11 этой книги полностью посвящена Behavior-Driven Development. В ней рассматриваются рецепты использования Cucumber-JVM (Java), SpecFlow (.NET), Capybara (Ruby) и Behave (Python) с Selenium WebDriver. Отличный ресурс для понимания кросс-платформенных аспектов BDD .

- **Manning Publications. "BDD in Action: Behavior-Driven Development for the whole software lifecycle" [Электронный ресурс] / liveBook.**
  - **Доступ (глава):** https://livebook.manning.com/book/bdd-in-action/chapter-6/
  - **Аннотация:** Глава 6 "Automating the scenarios" из этой фундаментальной книги объясняет принципы автоматизации сценариев, ответственность step definitions и демонстрирует реализацию на Java (JBehave, Cucumber-JVM), Python (Behave), .NET (SpecFlow) и JavaScript (Cucumber-JS) .

- **North, D. "Introducing BDD" [Электронный ресурс] / Dan North & Associates.**
  - **Доступ:** http://dannorth.net/introducing-bdd/
  - **Аннотация:** Оригинальная статья Дэна Норта, в которой он впервые представил концепцию Behavior-Driven Development в 2003 году. Обязательна к прочтению для понимания философии и мотивов создания BDD .